页面替换算法设计

Demand paging 下的自适应预测式页面替换

One-Day Hackathon 汇报 · 10.29

目标：用最少的缺页，处理没见过的访问序列

规则

- 只做 demand paging ，不预取
- 每次缺页只换入当前页
- 页框 5/10/20/50/100
- 每档从空内存开始

评分

- 现场未知 trace
- 五档缺页次数的算术平均
- 越少越好

难点

- MIN 要读未来，现实做不到
- 经典算法只看一种信号
- 访问模式因程序而异

我们的思路

只用已经发生的访问历史，估计每个驻留页“下次还要多久才会被用到”，淘汰预计最晚被用到的页——在线地逼近 MIN 。

五种算法：它们靠什么选择受害页？

FIFO

依据

进入内存最久

做法

公平，但不管页面是否仍在使用

特点

最简单

LRU

依据

最近最久未访问

做法

假设最近用过的会再用

特点

循环访问超过内存时反而全错

Second chance

依据

FIFO + 访问位

做法

被访问过的页获得第二次机会

特点

只有 1 位信息，区分度低

MIN (OPT)

依据

未来最晚才访问

做法

理论最优，现场不可用

特点

需要读取未来

我们的算法

依据

预测下次访问时间

做法

间隔 + 上下文，多策略在线选择

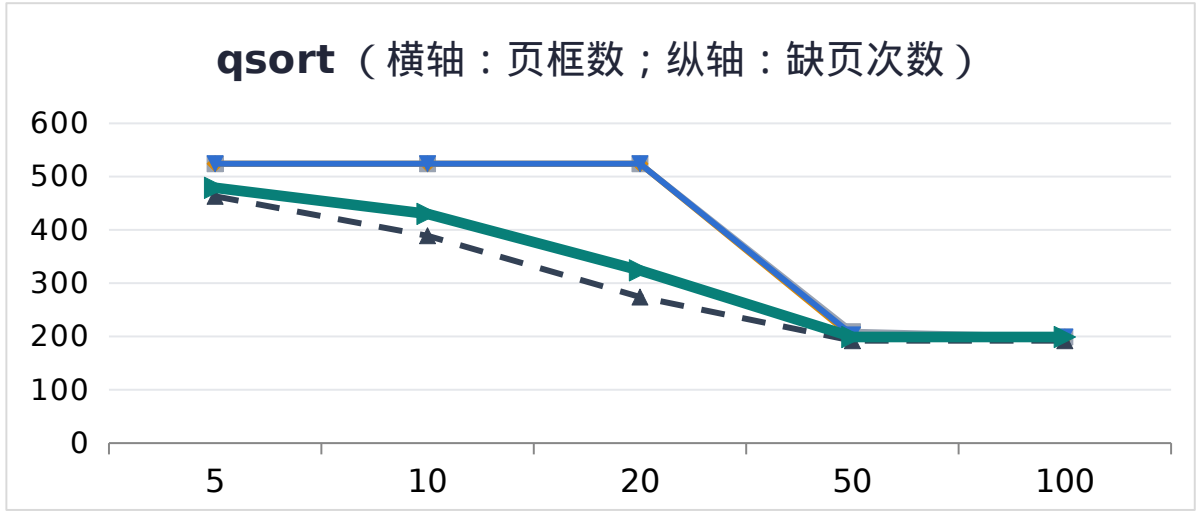
特点

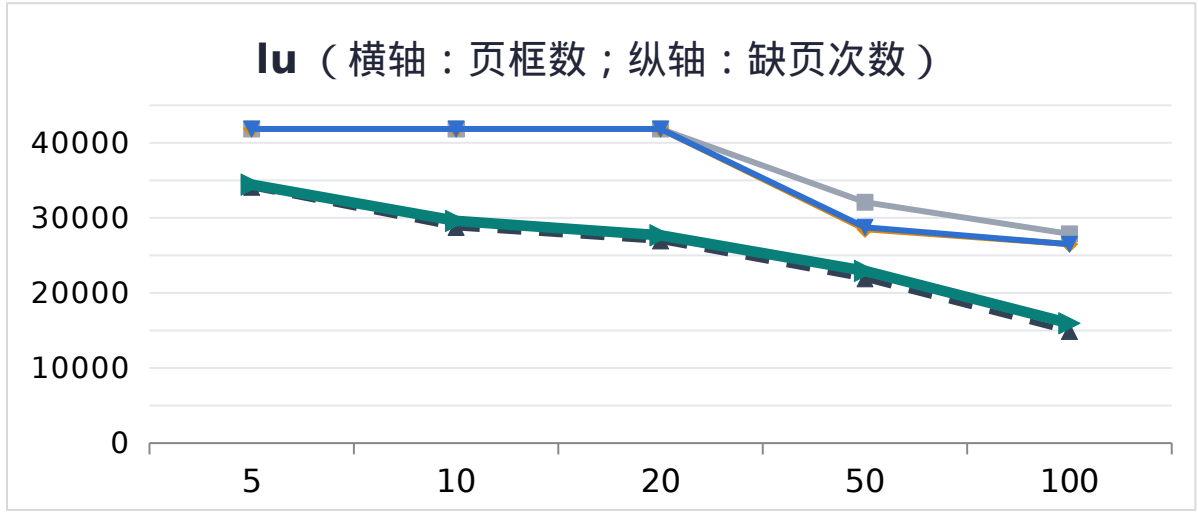
用计算换更少的缺页

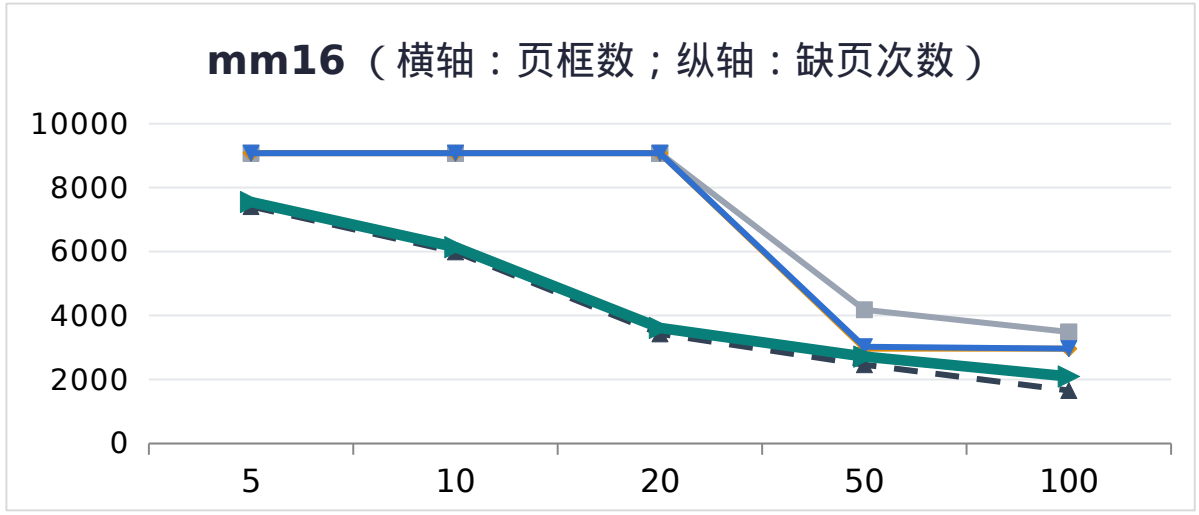
另外也实现了 Random、 LFU 作为参照；以下对比以这五种为主。

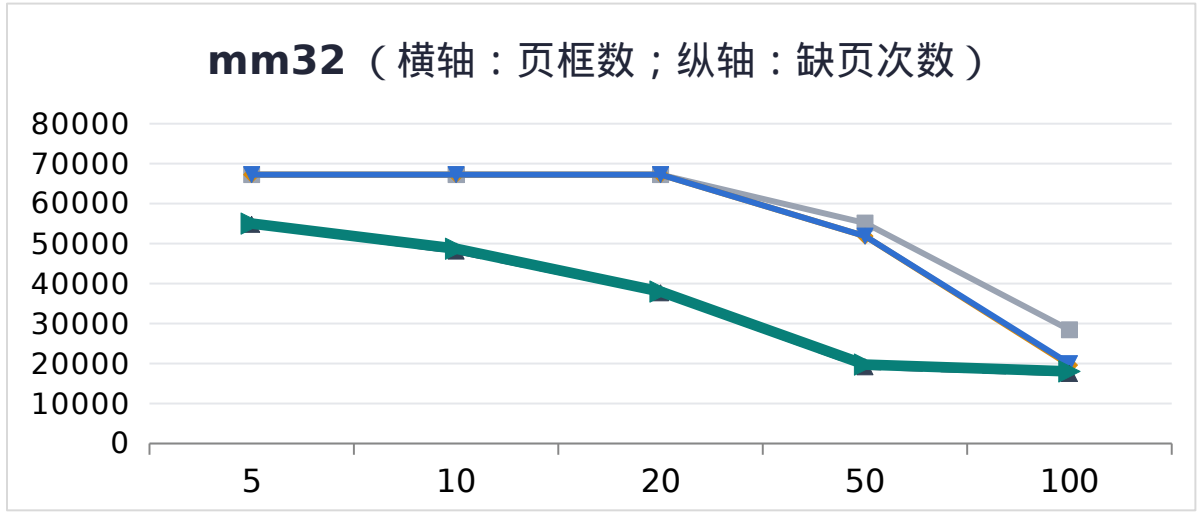
不同页框数下的缺页次数

FIFO　LRU　Second chance　MIN (OPT) 虚线　我们的算法



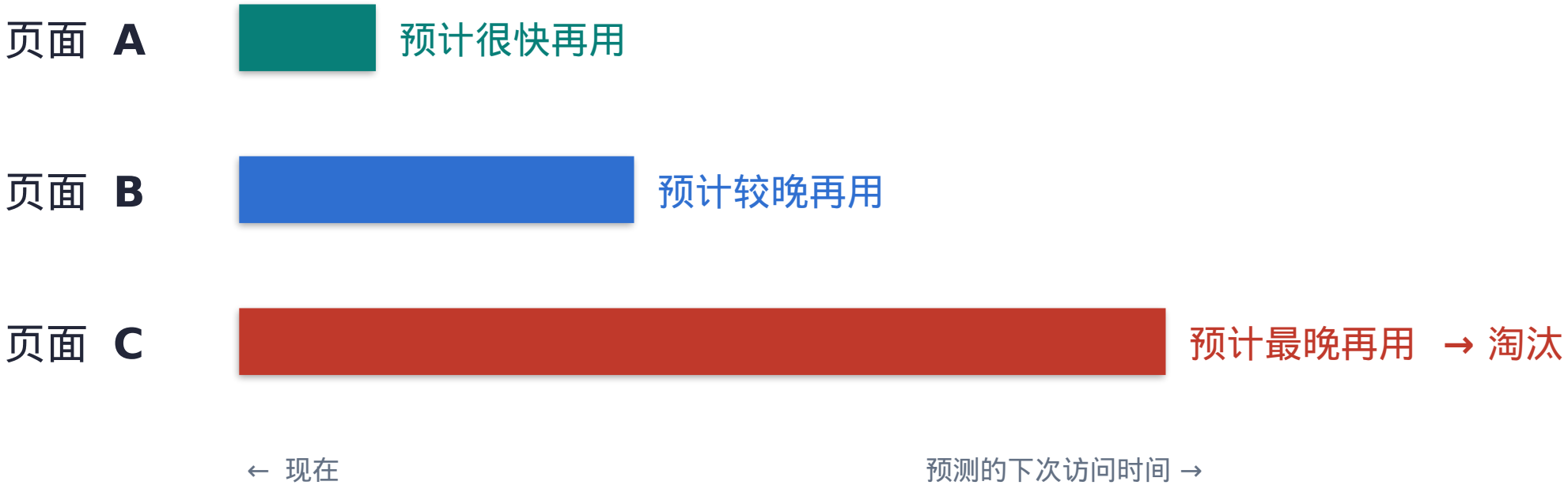






核心思想：预测“下次访问还有多久”

缺页发生，内存已满：谁最晚才会被再次用到？（示意）



为什么比 LRU 好：LRU 只看“上次什么时候用过”；我们看“这页通常隔多久回来”。循环访问时，刚用过的页恰恰最晚才被再用。

新信号：等待时间本身就是信息

只用平均值（容易错）

示意：历史间隔 10、100、10、100
平均 ≈ 55 —— 实际从不出现
页面已等待 20，仍按 55 预测

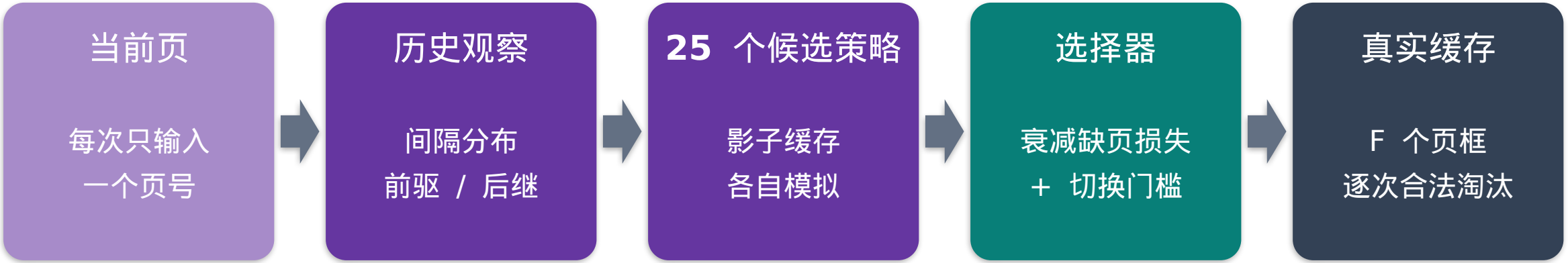
条件复用分布（我们）

已等待 20 → 排除“间隔 10”
只保留大于 20 的历史间隔
预测 ≈ 100

怎样实现（有界内存）

- 每组最多 16 个加权“间隔簇”，旧证据每次乘 0.98，适应访问模式变化
- 分组上下文：最近间隔、前后间隔对、前驱页、后继页
- 页面已被淘汰出历史表 → 记为“尚未返回”，加有限惩罚
- “后继页”指已经到达的当前页，不是偷看下一页

怎么落地：25 个候选策略在线竞争



候选策略

- LRU 、频率、随机
- 间隔预测、重放、条件分布、上下文 …

选择规则

- 损失 $L \leftarrow (1 - 1/W) \cdot L +$ 本次是否缺页
- $W = \max(32, 8F)$ ；领先超过 $W/20$ 才切换

遵守规则：没有预取，也没有偷看未来

✓ **只能 on-demand**

缺页才换入，每次只换入当前页，不预取

✓ **不看未来**

接口一次只收到当前页号，无文件句柄、无长度

✓ **真实页框不超限**

影子只含页号；真实缓存始终 ≤ F 页

✓ **不依赖文件名 / 页号大小**

页号已是页号，不再除以页大小

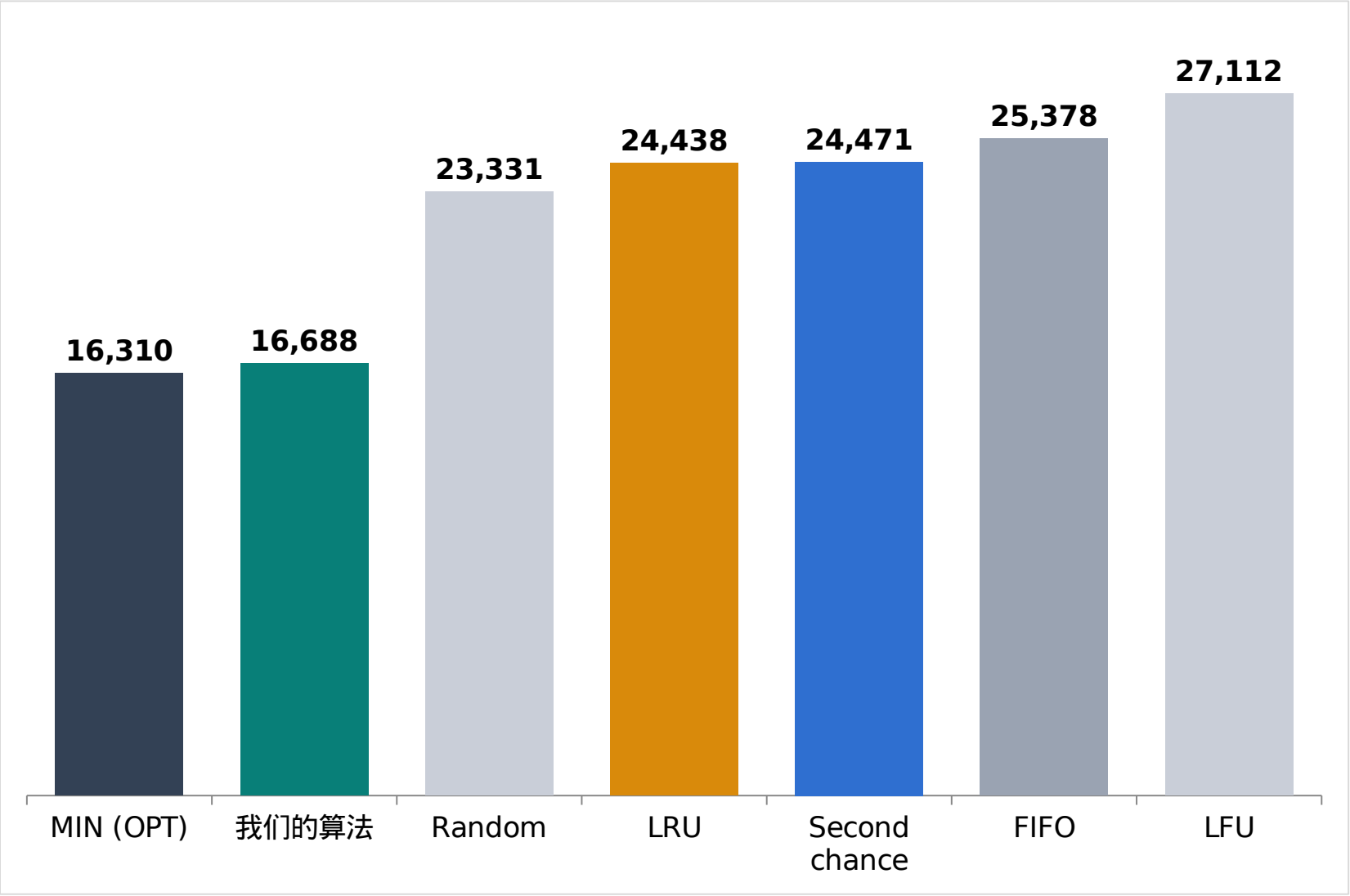
✓ **结果可复现**

随机算法使用 30 个固定种子；全部原始数据保留

✓ **测试通过**

Release + ASan + UBSan ；与精确 OPT 对照

总体结果：四条课程 trace × 五档页框的平均缺页



−31.7%

相对 LRU 的缺页减少

+2.3%

距离 MIN 理论下界

基本算法约 2.4–2.5 万
我们 16,688

不同访问序列：每条 **trace** 的平均缺页

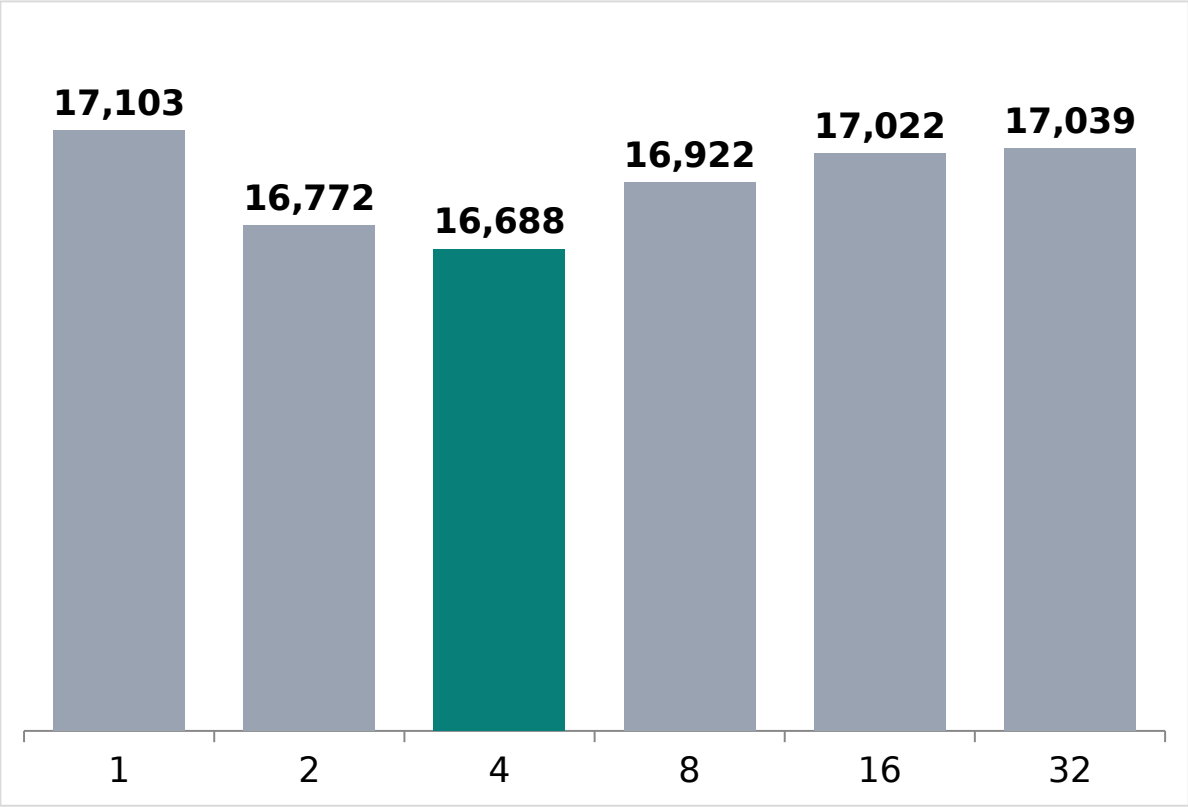
Trace	FIFO	LRU	Second chance	MIN (OPT)	我们的算法	较最佳经典算法少
qsort	396.0	394.0	394.8	302.0	**326.2**	17.2%
lu	37,086.6	36,093.6	36,143.0	25,293.8	**26,115.2**	27.6%
mm16	6,978.4	6,630.6	6,642.4	4,188.2	**4,421.2**	33.3%
mm32	57,052.4	54,633.8	54,703.4	35,455.2	**35,891.2**	34.3%

数字为五档页框（5、10、20、50、100）缺页次数的平均值。

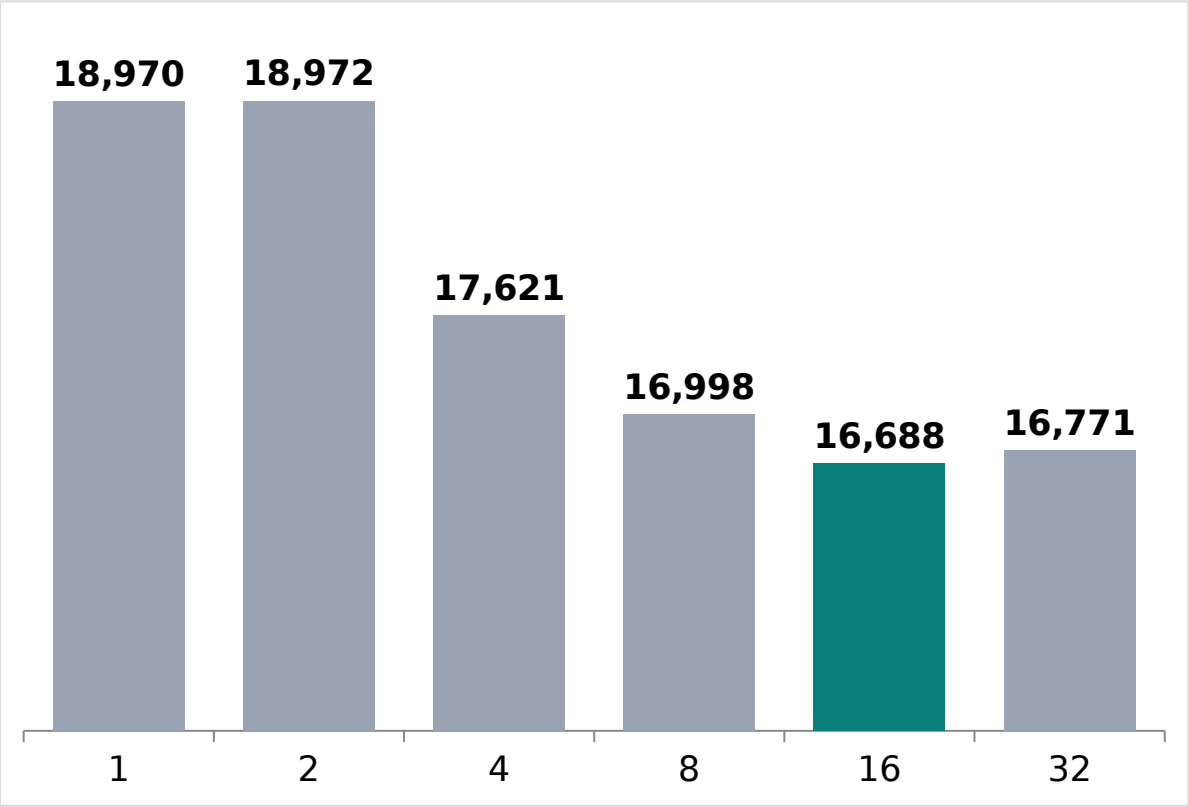
完整的逐页框数字见附录。

算法参数的影响

窗口系数（选择器比较窗口）



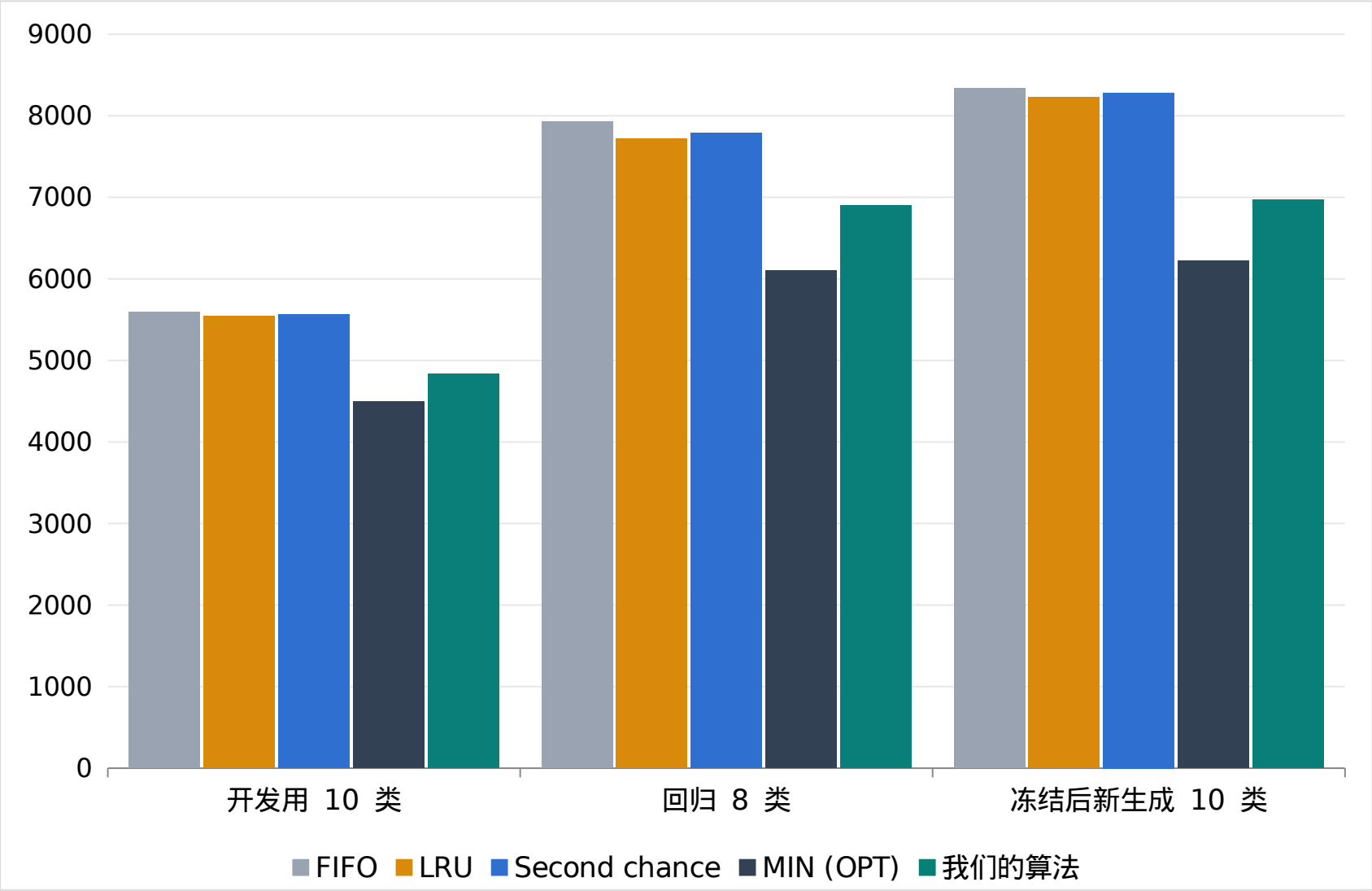
历史系数（每页历史容量）



纵轴：四条课程 trace 的平均缺页次数（已截断起点）；绿色为我们采用的取值。

默认：窗口系数 = 4，历史系数 = 16。

换一批序列：构造的循环、热点、 Zipf 、混合程序



新构造序列

−15.4%

相对 LRU

+12.0%

距离 MIN

这 10 类序列在算法冻结后才生成，未用于调参；个别序列略差于 LRU（如 matrix_ikj_24），已如实记录。

开销与局限

代价

- mm32（ 67,235 次访问）五档合计约 4.5 秒
- 一百万行纯扫描：峰值内存约 20 MiB
- 内存随页框数有界，与输入长度无关

局限

- 依赖访问模式有重复
- 完全随机、第一次出现的页无法预测
- 不保证任意未知输入都第一

现场使用

直接读取老师给的文件，一行一个页号，输出五档缺页次数和平均值。

演练（ mm32 ）： 55,006 · 48,712 · 37,986 · 19,730 · 18,022 ，平均 35,891.20

总结

1

五种算法完整对比

FIFO · LRU · Second chance · MIN · 我们的算法； 5 种页框、 4 条 trace 、多组参数

2

核心贡献

条件复用分布预测下次访问时间； 25 个候选策略在线竞争，真实页框始终合法

3

结果

课程 trace 平均缺页比 LRU 少 32% ，仅比理论下界 MIN 多 2.3% ；新构造序列仍领先

4

诚实说明

依赖访问模式重复；未知文件不保证第一；四条课程 trace 参与过开发

附录：**qsort** 完整缺页次数

页框数	Random	FIFO	MIN (OPT)	LFU	LRU	Second chance	我们的算法
5	523.1	524	463	524	524	524	**479**
10	502.5	524	389	524	524	524	**430**
20	389.8	524	274	524	524	524	**324**
50	217.7	209	192	199	199	203	**199**
100	196.3	199	192	199	199	199	**199**

附录：`lu` 完整缺页次数

页框数	Random	FIFO	MIN (OPT)	LFU	LRU	Second chance	我们的算法
5	41,708.4	41,832	34,076	41,832	41,832	41,832	**34,426**
10	40,317.5	41,832	28,722	41,832	41,832	41,832	**29,594**
20	36,796.5	41,832	26,922	41,832	41,832	41,832	**27,672**
50	30,923.2	32,073	21,880	34,653	28,475	28,736	**22,930**
100	23,802.3	27,864	14,869	32,969	26,497	26,483	**15,954**

附录：mm16 完整缺页次数

页框数	Random	FIFO	MIN (OPT)	LFU	LRU	Second chance	我们的算法
5	9,046.6	9,075	7,407	9,075	9,075	9,075	**7,548**
10	8,625.4	9,075	5,990	9,075	9,075	9,075	**6,136**
20	6,922.1	9,075	3,426	9,075	9,075	9,075	**3,612**
50	4,292.1	4,180	2,461	6,642	2,964	3,023	**2,715**
100	2,964	3,487	1,657	6,077	2,964	2,964	**2,095**

附录：mm32 完整缺页次数

页框数	Random	FIFO	MIN (OPT)	LFU	LRU	Second chance	我们的算法
5	67,071.2	67,235	54,807	67,235	67,235	67,235	**55,006**
10	65,301	67,235	48,109	67,235	67,235	67,235	**48,712**
20	59,191.5	67,235	37,682	67,235	67,235	67,235	**37,986**
50	39,745.6	55,136	19,164	53,805	51,860	51,865	**19,730**
100	28,090.6	28,421	17,514	51,699	19,604	19,947	**18,022**